# Raspberry Pi Video Player

## Technical User Manual

Stage Audio Works

**Applies to software version v16**

Raspberry Pi 5 | Raspberry Pi OS Bookworm 64-bit
Web interface on port 5000 | UDP control on port 5505
Document generated 2026-09-22

This software is provided with no warranty and no support.
Units have no authentication — keep them on a restricted network.

# Contents


**1. What the system does** 4

**2. Hardware requirements** 4

2.1 Button wiring . . . 5

2.2 Display . . . 5

**3. Video content requirements** 5

**4. Everyday operation — the web interface** 6

4.1 Finding the unit . . . 6

4.2 Player name . . . 6

4.3 Playback controls . . . 6

4.4 Volume . . . 7

4.5 Override mode . . . 7

4.6 Managing videos . . . 7

4.7 Output resolution and refresh rate . . . 8

4.8 Audio output . . . 8

4.9 System panel . . . 8

4.10 Logs . . . 9

4.11 Software update . . . 9

4.12 Branding . . . 10

**5. The physical buttons** 10

**6. Installing a new unit** 10

6.1 Units with internet access . . . 10

6.2 What the installer does . . . 10

6.3 After installing . . . 11

**7. Updating an existing unit** 11

7.1 Through the web interface . . . 11

7.2 Units with internet access . . . 11

7.3 Units behind a restrictive firewall . . . 11

**8. Network control over UDP** 12

**9. HTTP API reference** 12

Playback . . . 12

Content . . . 12

Output . . . 13

System . . . 13

**10. Files, ports and services** **14**

**11. Diagnostics and fault-finding** **14**

11.1 Check the power supply first . . . 14

11.2 Confirm which software is actually running . . . 14

11.3 Standard checks . . . 14

11.4 Symptom table . . . 15

11.5 When reporting a fault . . . 15

**12. Limitations and known issues** **15**

**13. AES67 network audio (optional)** **16**

13.1 What it does . . . 16

13.2 Requirements . . . 16

13.3 Installing . . . 17

13.4 Using it . . . 18

13.5 Lip-sync . . . 19

13.6 Updating . . . 19

13.7 Files, ports and services . . . 19

13.8 Fault-finding . . . 20

13.9 HTTP API . . . 20

13.10 Limitations . . . 21

**14. Bluetooth audio** **21**

14.1 Before you start . . . 21

14.2 Setting it up . . . 21

14.3 What happens on its own . . . 22

14.4 Turning it off, forgetting a device . . . 22

14.5 What the panel says . . . 22

14.6 Fault-finding . . . 23

14.7 Files and API . . . 23

**Applies to:** software version **v16**. Features that arrived in a recent version say which, so a unit running an older version can be told apart. **Last updated:** 2026-10-26

This manual covers day-to-day operation, installation, network control, the HTTP API, and fault-finding. Sections 2–5 are what an on-site operator needs; sections 6 onwards are for whoever installs and maintains the units.

The web interface opens this manual itself: **User manual** at the top of the page, and **? Help** on each panel, which opens the section about that panel (v14). It opens the copy installed on the unit, so it works without internet access; **latest online** opens the current version on GitHub.

# 1. What the system does

A headless video player for museum, exhibition and event installations, running on a Raspberry Pi 5 with no monitor, keyboard or mouse attached after installation.

**Normal behaviour:**

- An **attractor loop** plays continuously whenever the unit is idle.
- A visitor presses one of up to **three buttons**. The matching clip plays once, then the attractor loop resumes.
- **Override mode** replaces the attractor loop with a different video and ignores the buttons entirely. Intended for closures, private functions and maintenance.

Everything is managed from a web page served by the Pi itself. The unit is also controllable across the network by simple UDP commands, for integration with show control systems.

**The five video slots**, all stored in `/home/pi/videos/`:

| Slot | Filename | Plays when |
|---|---|---|
| Loop | `loop.mp4` | Always, when idle. This is the attractor. |
| Trigger 1 | `trigger1.mp4` | Button on GPIO 17, or `TRIGGER_1` over UDP |
| Trigger 2 | `trigger2.mp4` | Button on GPIO 27, or `TRIGGER_2` over UDP |
| Trigger 3 | `trigger3.mp4` | Button on GPIO 22, or `TRIGGER_3` over UDP |
| Override | `override.mp4` | While Override mode is switched on |

A trigger press is **ignored**, not queued, when any of the following is true:

- Override mode is active
- A trigger clip is already playing
- The player is mid-restart (for example, just after an audio or resolution change)

Each ignored press is written to the log with the reason.

# 2. Hardware requirements

| Item | Requirement |
|---|---|
| Computer | Raspberry Pi 5. 2 GB is sufficient — the player uses roughly 400 MB in operation. |
| Power supply | **5 V / 5 A official supply.** See the warning below. |
| OS | Raspberry Pi OS 64-bit, Bookworm or Trixie. The **Lite** version — the desktop version's PipeWire takes over Bluetooth audio. |

| Item | Requirement |
|---|---|
| User account | `pi` |
| Display | Connected to **HDMI port 0** — the socket nearest the USB-C power connector |
| Network | Wired Ethernet strongly preferred for exhibition installs |
| Buttons | Momentary, normally-open, wired between the GPIO pin and ground |

### ! Power supply

The Pi 5 needs a **5 V / 5 A** supply. An underpowered unit boots and idles perfectly, then crashes or behaves erratically the moment video decoding loads the GPU. **It looks exactly like a software fault.**

Before investigating any strange behaviour, check for undervoltage first. The web interface's System panel warns of it (v13), or over SSH:

```
dmesg | grep -i volt
```

## 2.1 Button wiring

Buttons connect between the GPIO pin and any ground pin. The inputs use the Pi's internal **pull-up** resistors, so no external components are required — pressing the button pulls the pin to ground and that falling edge fires the trigger.

| Trigger | BCM GPIO | Physical pin |
|---|---|---|
| Trigger 1 | GPIO 17 | Pin 11 |
| Trigger 2 | GPIO 27 | Pin 13 |
| Trigger 3 | GPIO 22 | Pin 15 |

Ground is available on physical pins 6, 9, 14, 20, 25, 30, 34 and 39.

Inputs are polled at 50 Hz with a **500 ms debounce**, so a single press cannot register twice, and holding a button down does not repeat.

## 2.2 Display

The player drives the display directly through DRM/KMS — there is no desktop environment running. The display should be **powered on and connected before the Pi boots**, so the Pi can read its capabilities.

# 3. Video content requirements

| Property | Requirement |
|---|---|
| Container | **MP4 only.** Other file types are rejected on upload. |
| Frame rate | **25 fps (PAL).** Content for these installations is PAL region. |
| Resolution | **1920×1080 recommended.** See the note below on 4K. |
| Audio | Any; the player downmixes surround to stereo automatically. |

### Encode at 1080p, not 4K

4K H.265 playback drops frames on the Pi 5. Hardware decoding keeps up fine, but every decoded frame has to be copied from GPU memory to system RAM for the display path, in software, at roughly 200 MB/s — and that is the bottleneck. Measured on hardware: zero decoder drops, but 38 dropped frames at the display stage and mpv sitting at 184% CPU.

**1080p is visually indistinguishable at normal exhibition viewing distances and runs with enormous headroom.** If 4K is a hard requirement, re-test periodically — this may improve with future kernel and firmware updates.

### Prefer H.265 (HEVC) to H.264

The Pi 5 decodes **H.265 in hardware**, but has **no H.264 decoder at all**: H.264 is decoded in software on the CPU. A 1080p H.264 clip still plays without dropped frames — measured on the bench at about 175% CPU — but that leaves far less headroom, and a hotter Pi. Encode new content as **H.265, 1080p, 25 fps**.

Because content is 25 fps, a **50 Hz** display mode divides evenly into the frame rate and a 60 Hz one does not. If a clip judders on an otherwise clean file, forcing 50 Hz is the first thing to try (see section 4.7).

# 4. Everyday operation — the web interface

## 4.1 Finding the unit

Open a browser on the same network and go to:

```
http://<pi-ip-address>:5000
```

**Reading the address off the screen.** After the unit boots, its console stays on the display for **30 seconds** before the video starts (v15). Above the login prompt it shows the IP address and the web interface's address to type, for example `Stage Audio Works Video Player on gallery-2 - web UI: http://<its IP address>:5000`. No login is needed. The web interface is already up during those 30 seconds. A restart of the player alone (an update, say) does not wait; only a boot does.

If the unit has been given a hostname, `http://<hostname>.local:5000` may also work, depending on the network.

**There is no password.** Anyone who can reach the unit on the network can control it, replace its content, reboot it, or replace its software. Put these units on a restricted VLAN or an isolated installation network. See section 12.

## 4.2 Player name

The field at the top of the page names this unit — typically its physical location, for example *"Gallery 2 — East Wall"*. Type a name and press **Save**.

The name persists across reboots and is shown in the page title, which makes it far easier to tell several open browser tabs apart when managing a fleet.

## 4.3 Playback controls

| Control | Effect |
|---|---|
| **Play** | Resumes playback after a pause |
| **Pause** | Freezes the current frame |
| **Restart Loop** | Jumps back to the start of the attractor loop |
| **Reboot Pi** | Reboots the whole unit. Takes 30–60 seconds to return. |

Pause is a genuine pause — the frame stays on screen. It does not survive a reboot.

**"No screen connected."** When the status line says this, the Pi cannot see a screen on HDMI port 0 — it is switched off, unplugged, or on the other port. Many screens in standby look unplugged to the Pi. Nothing is wrong with the player: the video starts by itself within a few seconds of the screen being switched on or plugged in (v16). The log says so once rather than every few seconds.

## 4.4 Volume

The volume slider runs 0–100 and takes effect immediately. The setting **persists across reboots**.

Note this is the player's own volume. If the unit feeds an external amplifier or DSP, set this to a sensible fixed level and do the final level-setting downstream.

## 4.5 Override mode

Tick **Activate Override** to switch the unit to `override.mp4` immediately and ignore all button presses. Untick it to return to the attractor loop.

The override state **persists across reboots** — a unit switched to override on Friday is still in override on Monday. This is deliberate, so a closure survives a power cut, but it is also the single most common cause of *"the buttons have stopped working"*. Check the override tick box first.

Switching override on **interrupts a trigger clip that is already playing**, cleanly and without a visible stutter.

## 4.6 Managing videos

Each slot has its own row showing a badge with the current file's size, and its modification date on hover. A slot with no file shows **missing**.

| Button | Effect |
|---|---|
| **Upload** | Replaces that slot's video. A progress bar shows transfer status. |
| **Download** | Downloads the current file, so you can retrieve what is on a unit |
| **Delete** | **Deletes** the clip. Greyed out when the slot is already empty. |

Deleting always asks for confirmation first — there is no second copy of a clip on the Pi, and no undo.

**Deleting the clip that is currently on screen does not black out the display.** The player keeps the file open and continues looping it until it next restarts, so an exhibit carries on running while you upload a replacement. The confirmation message tells you when this has happened.

> **Deleting `loop.mp4` is the one genuinely destructive case.** Every other slot degrades harmlessly when empty — a missing trigger clip is simply logged and ignored, and a missing override falls back to the loop. But with no attractor loop there is nothing to play, so the screen goes black the next time the player restarts. The confirmation dialog warns about this specifically.

**Upload size limit.** Recalculated on every request as 85% of currently free disk space, with a 64 MB floor. The figure is shown above the upload rows. Uploads are streamed to disk rather than held in memory, so a 300 MB video does not need 300 MB of RAM.

## 4.7 Output resolution and refresh rate

Two dropdowns force the HDMI output mode. Both settings **persist across reboots**.

- **Resolution** — lists only the modes the attached display reports over its EDID, plus *Auto*, which uses whatever the display says it prefers. Auto is the default and is what every unit runs unless you change it.
- **Refresh rate** — Auto, 24, 25, 30, 50 or 60 Hz. Disabled when Resolution is set to Auto, because a refresh rate can only be pinned to a specific mode.

Press **Apply**. The player restarts to take up the new mode, so **the picture goes black for roughly a second.** The status line then reports the active mode and the refresh rate the player measured.

> **The display does not report which refresh rates it supports for each resolution — only the resolutions.** The rates offered are therefore candidates, not a verified list, and some combinations will not exist on a given screen.
>
> This is safe to experiment with: if the player cannot start on the mode you chose, it automatically falls back to the display's preferred mode, saves that, and restarts. You get a message saying the mode was rejected rather than a black screen.

Remember the PAL note from section 3: for 25 fps content, **50 Hz** is the mode worth trying if anything judders.

## 4.8 Audio output

Lists every ALSA playback device the Pi can see. The default is HDMI audio on port

0. Plug in a USB audio adapter, press **Refresh**, select it, and press **Apply**.

The player restarts to pick up the new device, so there is a brief interruption. The selection persists across reboots.

> USB audio adapters can change their ALSA card name between reboots if other USB devices are added or removed. If audio silently moves to the wrong output after a hardware change, re-select the device here.

On a unit with AES67 network audio installed (v12), the list also offers **AES67 network** — see section 13. With Bluetooth turned on (v12), it also lists paired Bluetooth speakers — see section 14.

## 4.9 System panel

Live readout, refreshed every five seconds:

- CPU temperature
- Memory used and available
- Disk space used and available
- System uptime
- Hostname
- **RAM-pinned clips** (v12): which clips are locked into memory. The player keeps the loop — then the override, then the triggers — in RAM where there is room, so playback carries on even if the SD card turns slow or starts failing. Pinned clips count as used memory, so a well-filled memory bar is normal. *Limited* means the service file is older than the player: run the one-line installer (section 7.2) or `sudo bash update.sh`

`--service`.

It also shows a warning above these, only when there is something to say (v13): **under-voltage now** (red) or **under-voltage since the last boot** (amber). Either means the power supply cannot keep up — see section 11.1.

Worth a glance during commissioning: a unit sitting at a high idle temperature or low free disk space will cause trouble later.

## 4.10 Logs

**Show Logs** expands a live log viewer that refreshes every five seconds. You can also:

- **Change the log level** — `DEBUG`, `INFO`, `WARNING`, `ERROR`, `CRITICAL`. `INFO` is the normal setting. `DEBUG` is very verbose and includes mpv's own output; use it while diagnosing, then set it back.
- **Download** the full log file, for sending on
- **Wipe** the log, to get a clean baseline before reproducing a fault

Logs rotate automatically at 200 MB and older files are compressed, keeping up to 10 of them, so the log cannot fill the disk.

## 4.11 Software update

The **Software Update** panel shows the installed version, and updates the player software through the browser — no SSH needed. Two ways:

**Update from GitHub** (v12). Press **Check for updates**. If a newer version has been published, it says so and offers **Update now**: the unit downloads it, checks its checksum, validates it as below and installs it, keeping the current version as a backup. A unit already running v15 or later also brings this manual up to date, checked the same way; if only the manual fails, the player is still updated. (The update is done by the version being replaced, so updating *to* v15 does not bring the manual yet — run the one-line installer once, section 7.2.) The manual only travels with a new version: if the version has not changed, the unit keeps its copy. The page reloads by itself once the new version is running. It never installs an older version over a newer one, and needs internet access to GitHub — on a closed network the check says so and nothing changes.

It updates **the player program only**. When a release also changes the service file or the installer, the check says so; run the one-line installer afterwards (section 7.2).

**Upload a script manually.** Choose a `.py` file and press **Deploy Script**. Before anything is replaced, the upload is validated:

1. It must be valid UTF-8 text
2. It must parse as valid Python
3. It must contain the `SAW_PLAYER_SIGNATURE` identifier
4. It must contain all of the required function names

A file failing any check is rejected and nothing changes. On success the current script is backed up to `/home/pi/saw_player.py.bak`, the new one is written, and the service restarts. Video drops out for a few seconds.

**If a deployed script fails to start, the web interface goes with it** — so it cannot rescue itself. Recovery needs SSH:

```
sudo cp /home/pi/saw_player.py.bak /home/pi/saw_player.py && sudo systemctl restart saw-player
```

Have that ready before deploying to a unit you cannot easily reach.

### 4.12 Branding

If `/home/pi/saw.jpg` exists, it is displayed in the page footer. The check runs **once at startup**, so restart the service after copying the file:

```
sudo systemctl restart saw-player
```

If it does not appear, verify the file actually transferred as an image — a failed transfer that produces a zero-length or corrupt file is the usual cause:

```
file /home/pi/saw.jpg     # should say "JPEG image data"
```

## 5. The physical buttons

Momentary buttons wired as in section 2.1. One press plays the matching clip once, then the attractor loop resumes automatically.

- Presses are **ignored, not queued**, while a clip is already playing.
- All presses are ignored while Override mode is on.
- A 500 ms debounce means a single press cannot double-fire.

**If the buttons appear dead**, check in this order:

1. Is **Override mode** on? It ignores all buttons by design. This is the most common cause by a wide margin.
2. Does the trigger clip exist? A missing file logs a warning and does nothing.
3. Is a clip already playing? Presses during playback are intentionally ignored.
4. Check the wiring — button between the GPIO pin and ground.

The log records every press and, when ignored, the reason.

## 6. Installing a new unit

Network configuration — hostname and IP address — is **not** handled by the installer. Set those with Raspberry Pi Imager when writing the card, with a DHCP reservation on the site network, or with `raspi-config` / `nmcli` afterwards.

### 6.1 Units with internet access

One command installs everything and starts the player:

```
curl -fsSL https://raw.githubusercontent.com/stageaudioworks/pi-video-player-dist/main/bootstrap.sh | su
do bash
```

To reboot when it finishes:

```
curl -fsSL https://raw.githubusercontent.com/stageaudioworks/pi-video-player-dist/main/bootstrap.sh | su
do bash -s -- --reboot
```

This clones the project to `/home/pi/pi-video-player` and runs the installer.

### 6.2 What the installer does

1. Installs dependencies: **mpv**, Python packages and the Bluetooth packages
2. Creates `/home/pi/videos/`, installs the player script and the systemd service

3. Configures the Raspberry Pi hardware watchdog, so a hung unit reboots itself. systemd itself keeps it fed. (Up to v12 the installer also ran a separate watchdog daemon, which held every boot back by about 90 seconds before the player started; from v13 the installer removes it.)
4. Writes the sudo rules the web interface needs for reboot and script update
5. Puts this manual on the unit as `/home/pi/saw_manual.pdf`, for the web interface's **User manual** link (v14)
6. Adds the web interface's address to the console, above the login prompt (v15) — see section 4.1
7. Starts the service

It is **non-interactive and safe to re-run** — every dependency is checked before installing, so a re-run is quick.

```
sudo bash install.sh              # install or re-install, then start
sudo bash install.sh --reboot     # ...and reboot when finished
sudo bash install.sh --uninstall  # remove the service and watchdog configuration
sudo bash install.sh --help
```

Uninstalling leaves your videos, logs and the player script in place.

### 6.3 After installing

1. Upload at least `loop.mp4` through the web interface — with no attractor loop there is nothing to display.
2. Set the **Player name** to the unit's location.
3. Check audio output and, if needed, resolution and refresh rate.
4. **Reboot and confirm the unit comes up cleanly from cold.** This is the test that matters for an unattended installation, and it is easy to skip.

## 7. Updating an existing unit

Three routes, in order of convenience:

### 7.1 Through the web interface

Section 4.11: **Check for updates**, then **Update now**. Best for a single unit when you already have the browser open. It updates the player program only; for changes to the installer or service file, use 7.2 as well.

### 7.2 Units with internet access

**Exactly the same command as the install.** It pulls the latest code and skips anything already present:

```
curl -fsSL https://raw.githubusercontent.com/stageaudioworks/pi-video-player-dist/main/bootstrap.sh | su
do bash
```

### 7.3 Units behind a restrictive firewall

Some units sit on corporate networks that block package and repository access.

`update.sh` makes **no network calls whatsoever** — copy the project files across by SCP and run it locally:

```
sudo bash update.sh              # copy changed files and verify configuration
sudo bash update.sh --script     # player script only
sudo bash update.sh --service    # systemd service file only
sudo bash update.sh --setup      # configuration and sudo rules only, no file copy
sudo bash update.sh --restart    # restart the service only
```

It compares checksums and does nothing if the unit is already current, so it is safe to run against every unit in a fleet without worrying about needless restarts.

# 8. Network control over UDP

The player listens on **UDP port 5505**. Send the command as a plain text payload, with no terminator.

| Command | Effect |
|---|---|
| `PLAY` | Resume playback |
| `PAUSE` | Pause on the current frame |
| `RESTART_LOOP` | Restart the attractor loop from the beginning |
| `OVERRIDE_ON` | Switch to the override video, ignore buttons |
| `OVERRIDE_OFF` | Return to the attractor loop |
| `TRIGGER_1` | Fire trigger 1, exactly as a button press |
| `TRIGGER_2` | Fire trigger 2 |
| `TRIGGER_3` | Fire trigger 3 |

UDP triggers obey the same rules as physical buttons — ignored during override or while a clip is playing.

In the examples, replace `<unit-ip>` with the unit's IP address.

**From Linux or macOS:**

```
echo -n "TRIGGER_1" | nc -u <unit-ip> 5505
```

**From Windows PowerShell:**

```
$u = New-Object System.Net.Sockets.UdpClient
$b = [Text.Encoding]::ASCII.GetBytes("TRIGGER_1")
$u.Send($b, $b.Length, "<unit-ip>", 5505)
$u.Close()
```

There is no acknowledgement and no authentication. UDP delivery is not guaranteed; for critical cues, send the command and verify the result through `/status`.

# 9. HTTP API reference

All endpoints are served on port 5000. No authentication.

## Playback

| Method | Endpoint | Purpose |
|---|---|---|
| `GET` | `/status` | Current video, play state, position, remaining, override, volume |
| `POST` | `/play` | Resume |
| `POST` | `/pause` | Pause |
| `POST` | `/restart` | Restart the attractor loop |
| `POST` | `/set_override` | Body `{"active": true\|false}` |
| `GET/POST` | `/volume` | Get or set volume, 0–100 |

## Content

| Method | Endpoint | Purpose |
|---|---|---|
| `POST` | `/upload/loop`, `/upload/override`, `/upload/trigger/<1-3>` | Upload a clip (multipart, field `file`) |
| `GET` | `/download/loop`, `/download/override`, `/download/trigger/<1-3>` | Download a clip |
| `POST` | `/delete/loop`, `/delete/override`, `/delete/trigger/<1-3>` | Delete a clip |
| `GET` | `/system/video_slots` | Size, modification date and presence of each slot |
| `GET` | `/system/max_upload` | Current upload size limit in bytes |

## Output

| Method | Endpoint | Purpose |
|---|---|---|
| `GET` | `/audio_devices` | List available ALSA devices |
| `GET/POST` | `/audio_device` | Get or set the active audio device |
| `GET` | `/display_modes` | Modes the attached display reports |
| `GET/POST` | `/resolution` | Get or set output mode, e.g. `1920x1080@50` |

## System

| Method | Endpoint | Purpose |
|---|---|---|
| `GET` | `/system/info` | CPU temperature, memory, disk, uptime, hostname, RAM-pinned clips, under-voltage |
| `GET/POST` | `/player_name` | Get or set this unit's name |
| `POST` | `/system/reboot` | Reboot the Pi |
| `POST` | `/upload/script` | Deploy a new player script (validated) |
| `GET` | `/update/version` | The installed version |
| `GET` | `/update/check` | Compare with the published release on GitHub |
| `POST` | `/update/apply` | Download, verify and install a newer release |
| `GET` | `/manual` | This manual: the copy on the unit, else GitHub's. `?section=aes67` (also `system`, `audio`, `resolution`, `videos`, `update`, `bluetooth`) opens that section; `?online=1` always opens GitHub's |
| `GET` | `/logs/tail` | Recent log lines |
| `GET` | `/logs/download` | Download the full log |
| `POST` | `/logs/wipe` | Clear the log |
| `POST` | `/logs/set_level` | Set the log level |

Example — fire trigger 2 and read the resulting state:

```
curl -s http://<unit-ip>:5000/status
curl -X POST -H "Content-Type: application/json" \
     -d '{"active":true}' http://<unit-ip>:5000/set_override
```

# 10. Files, ports and services

| Item | Location |
|---|---|
| Player script (running) | `/home/pi/saw_player.py` |
| Previous script (backup) | `/home/pi/saw_player.py.bak` |
| Videos | `/home/pi/videos/` |
| Log file | `/home/pi/saw_player.log` |
| Override state and player name | `/home/pi/config.json` |
| Volume | `/home/pi/.mpv_volume` |
| Audio device | `/home/pi/.mpv_audio_device` |
| Output resolution | `/home/pi/.mpv_resolution` |
| Branding image | `/home/pi/saw.jpg` |
| This manual | `/home/pi/saw_manual.pdf` |
| Console line with the web interface's address | `/etc/issue.d/saw-player.issue` |
| systemd service | `saw-player` |
| Player control socket | `/tmp/saw_mpv.sock` |
| Web interface | TCP 5000 |
| UDP control | UDP 5505 |

Useful service commands:

```
sudo systemctl status saw-player     # is it running?
sudo systemctl restart saw-player    # restart it
sudo systemctl stop saw-player       # stop it
journalctl -u saw-player -f          # follow the log live
journalctl -u saw-player -n 50 --no-pager
```

# 11. Diagnostics and fault-finding

## 11.1 Check the power supply first

```
dmesg | grep -i volt
```

Undervoltage on a Pi 5 produces symptoms that look exactly like software faults. Rule it out before spending time anywhere else. The System panel shows a warning when it has happened since the last boot (v13).

## 11.2 Confirm which software is actually running

```
grep -m1 'Video Player  (v' /home/pi/saw_player.py
```

Several hours were once lost on this project diagnosing a "GPIO fault" that turned out to be an older script still deployed. Confirm the version before debugging behaviour.

## 11.3 Standard checks

```
sudo systemctl status saw-player             # is the service running?
pgrep -a mpv                                 # is the player process alive?
ls -la /tmp/saw_mpv.sock                     # is its control socket present?
cat /sys/class/drm/card1-HDMI-A-1/status     # is the display connected?
```

```
ls -la /home/pi/videos/                    # do the videos exist?
df -h                                      # is the disk full?
vcgencmd measure_temp                      # CPU temperature
aplay -l                                   # audio devices
ip -s link show eth0                       # network error counters
```

## 11.4 Symptom table

| Symptom | Check in this order |
|---|---|
| **No picture at all** | The web interface's status says **no screen connected** → switch the screen on, check the cable is in HDMI port 0 (`cat /sys/class/drm/card1-HDMI-A-1/status` should say `connected`) → power supply (`dmesg \| grep -i volt`) → is the service running → does `loop.mp4` exist (without it the player waits for an upload, and says so in the log) → is mpv running |
| **Buttons do nothing** | Override mode is on (by far the most common) → trigger file missing → a clip is already playing → wiring between GPIO pin and ground |
| **Web page loads but every field shows a dash** | A JavaScript error is stopping the page. Open the browser console. Usually follows a bad script deployment — roll back with the `.bak` file. |
| **No audio** | Audio device selection → volume slider not at zero → the physical output the display or amplifier is actually using → `aplay -l` shows the device |
| **Video judders** | Content is 4K — re-encode at 1080p → try a 50 Hz output mode for 25 fps content → check CPU temperature for throttling |
| **Unit unreachable on the network** | Cable and switch port → `ip -s link show eth0` for errors → does it have an address → has DHCP moved it |
| **Web page will not load but video plays** | The service is running but Flask has failed — check `journalctl -u saw-player -n 50` |
| **Reboot button does nothing** | The sudo rules are missing. Re-run `sudo bash update.sh --setup`. |
| **Upload rejected** | File is not `.mp4` → file is larger than the displayed limit → disk is full |

## 11.5 When reporting a fault

Include: the player name and location, the output of `grep -m1 'Video Player (v' /home/pi/saw_player.py`, the downloaded log file, and the output of

`dmesg | grep -i volt`.

# 12. Limitations and known issues

**There is no authentication anywhere.** The web interface and the UDP port are both completely open to anyone who can reach the unit. That includes rebooting it, replacing its content, and replacing its software through the script update — which is arbitrary code execution by design. **These units must be on a restricted or isolated network.** Do not expose port 5000 or 5505 to the internet or to a general-purpose corporate network.

**4K content drops frames.** See section 3. Encode at 1080p.

**Refresh rates are candidates, not a verified list.** The display reports which resolutions it supports but not which refresh rates, so some combinations offered in the dropdown will not exist on a given screen. The player falls back automatically if it cannot start on the mode chosen. *Note: this automatic fallback has been implemented and reviewed but not yet deliberately exercised in the field, so treat a rejected mode as something to verify rather than assume.*

**Override state survives reboots.** Intentional, so a closure is not undone by a power cut — but it is the most common cause of "the buttons have stopped working".

**Deleting a clip is permanent.** There is no copy kept on the Pi and no undo. Download a clip before deleting it if it is the only copy.

**A failed script deployment takes the web interface down with it.** Recovery requires SSH and the `.bak` file. See section 4.11.

**USB audio adapters can change ALSA card name** between reboots if other USB devices change. Re-select the device if audio moves unexpectedly.

# 13. AES67 network audio (optional)

> **Applies to v12 and later.** Proven on the bench (October 2026) with a Q-SYS Core as the receiver and a PTP grandmaster on the same VLAN: clean audio, channel order, clip changes, a pulled network cable, reboots and power cuts. At a new site, prove it on the real network before opening: audio, a clip change, a reboot.

## 13.1 What it does

Sends the player's audio onto the network as an **AES67 stream** — for example into a Q-SYS Core — instead of out of HDMI or a USB adapter. The video still plays on HDMI as normal.

- The stream is **48 kHz, 24-bit (L24), 1 ms packets**, **stereo** (channels 1–2) or **mono** (left and right mixed onto channel 1).
- It is named after the unit's **Player name** and sent to a **multicast group of its own**, 239.1.x.y, worked out from the unit's network card. Two units never share one by accident.
- It **stays on the air continuously** — through clip changes, clips with no soundtrack, player restarts, and (from v13) while another audio output is chosen — carrying silence between sounds. Receivers therefore never lose the stream and have to find it again.
- Nothing on the network can freeze the picture. If the network clock or the audio software fails, the video plays on and the web interface says what is wrong.

It is **optional**. On a unit where it has not been installed, the only sign of it is the AES67 panel offering to install it (13.3): the audio option does not appear, and the player behaves exactly as before.

## 13.2 Requirements

| Item | Requirement |
|---|---|
| Pi | Raspberry Pi 5, **2 GB or more** |
| Software | Raspberry Pi OS 64-bit, Bookworm or Trixie (check the kernel with `uname -r`). Kernel **6.12** is proven with every receiver. Kernels 6.15 and later send audio in bursts that receivers with small buffers can stutter on; a Q-SYS Core played it cleanly on **6.18** (Trixie). |

| Item | Requirement |
|---|---|
| Network | **Wired Ethernet** on the same network (VLAN) as the receiver |
| Clock | A **PTP grandmaster** on that network — normally the Q-SYS Core — in the same PTP domain (0 unless set otherwise). The Pi follows it and never becomes the master. |
| Switches | **IGMP snooping with a querier**, or no snooping at all. Snooping without a querier is the classic cause of a stream that is sent but never arrives. |
| Install | **Internet access for 20–40 minutes**, once. Build one unit, then clone its SD card (13.3). |

## 13.3 Installing

Install the player first (section 6). There are two ways to install AES67, both on a unit with internet access, and both run the same installer.

**From the web interface.** Every unit shows an **AES67 Network Audio** panel below Audio Output. Press **Check this unit**: it checks internet access, the kernel version, memory, free disk space and the network connection, and says plainly what would stop the install. If nothing does, **Install AES67** appears. It asks for confirmation, then shows the installer's progress as it runs.

| Unit | What happens to the picture during the install |
|---|---|
| 3 GB of memory or more | The picture and the web page normally **keep running**. The player releases its RAM-pinned clips (section 4.9) to make room, and pins them again afterwards. The build works the Pi hard, though, so **the screen could still go dark**, or the page stop answering, until it finishes. |
| 2 GB | The player is **stopped** for the build: the screen and the web page **go dark for 20–40 minutes** and come back by themselves when it finishes. |

Either way, do it **outside opening hours**. When it finishes, the panel says so and offers **Reboot now**, which is recommended. If it fails, the end of its log is shown; the whole log is `/var/log/saw-aes67-install.log`.

Installing from the web interface needs the one-line installer (section 6.1) to have been run with this version, which puts the means in place. If the panel says it is not set up yet, run the one-liner again.

**Over SSH:**

```
sudo bash /home/pi/pi-video-player/install-aes67.sh --check    # what is here; changes nothing
sudo bash /home/pi/pi-video-player/install-aes67.sh            # install
sudo reboot
```

The installer stops the player while it builds — the screen goes dark — and starts it again when it finishes, whatever happens. It installs:

1. Merging Technologies' **RAVENNA kernel module**, which provides the AES67 sound card and follows the network's PTP clock
2. The **aes67-daemon**, which publishes the stream and announces it (SAP and mDNS)
3. **saw-aes67-bridge**, a small service that keeps the AES67 sound card fed continuously and corrects for the difference between the Pi's clock and the network's
4. The system settings the AES67 software needs: CPU governor set to *performance*, network and real-time limits, and the kernel **held** at its current version, because a kernel update removes the sound card until the installer is run again

It is **safe to re-run**. A healthy installation is left exactly as it is; only the bridge is rebuilt, and only if it has changed. Options:

| Option | Effect |
|---|---|
| `--check` | Report what is installed; change nothing |
| `--ptp-domain N` | PTP domain of the grandmaster, if not 0 |
| `--force-rebuild` | Rebuild everything even if healthy |
| `--keep-player` | Do not stop the player while building (risky on a 2 GB Pi) |
| `--keep-governor` | Leave the CPU governor alone — for fanless enclosures that run hot |
| `--no-hold-kernel` | Do not hold the kernel version |
| `--jobs N` | Compile N files at once (default: worked out from free memory) |
| `--uninstall` | Remove the AES67 software |

**Cloning.** Units behind a firewall cannot build this themselves. Build one unit with internet access, then image its SD card for the others. Each clone notices at first boot that it is on different hardware, takes its own multicast group and clears the stream it inherited — so clones never transmit on the original's group. Set each clone's Player name, since that is what its stream is called.

## 13.4 Using it

In **Audio Output** (section 4.8), choose **AES67 network** and press **Apply**. A new **AES67 Network Audio** panel shows the state in one line at the top:

| The panel says | Meaning and what to do |
|---|---|
| **On air** | The stream is being sent. The line gives its name, group and port. |
| Another audio output is chosen | The stream stays on air, silent, so receivers keep it (v13; v12 switched it off). Choose AES67 network again and the sound is back at once. |
| Not in use | AES67 has not been chosen as the output since it was installed. |
| Waiting for PTP: no grandmaster heard | No PTP clock on this network. Check the Core is the grandmaster, the Pi is on the right VLAN, and the PTP domain matches. |
| Waiting for PTP to lock | A grandmaster is heard but not yet followed. Usually a few seconds; if it persists, the network is not passing PTP cleanly. |
| The AES67 bridge is not running | `sudo systemctl status saw-aes67-bridge` |
| The AES67 daemon is not answering | `sudo systemctl status aes67-daemon` |
| The AES67 card cannot be opened | Usually a kernel update. Run `install-aes67.sh` again. |
| Setting up the stream / Opening the card / Starting | Normal for a few seconds after a change or a restart |

Below it: the stream's details, the clock correction in use, and counters for audio problems since the bridge started. **Show SDP** displays the stream description a receiver can import.

**Receiving it in Q-SYS.** In an AES67 receiver, **Auto** mode lists the stream by its name from its announcements — allow **up to 30 seconds** after the stream starts for it to appear: they are sent every 30 seconds. In **Manual** mode enter: the group shown in the panel, port **5004**, **L24**, **48 kHz**, packet time **1 ms** (48 samples), and **2 channels** (or 1 for mono).

**A receiver that shows the stream but plays nothing.** Seen on the bench with a Q-SYS Core after the stream went off the network and came back under the same name — after the aes67-daemon restarted, and (in v12)

after another output was chosen and AES67 chosen again. The packets arrive and are correct, but the Core stays attached to the stream as it was before, for 12–23 minutes. Its receiver's **Details** show `connected=0`. The fix is immediate: change the **Player Name** (it renames the stream) and choose the new name in the receiver. From v13 choosing another output no longer takes the stream off the air, so only a daemon restart can cause it — and the panel then shows an alert saying the daemon restarted, until dismissed.

**Changes that restart the stream.** Renaming the unit renames the stream, so a receiver that chose it by name has to choose it again. Switching between stereo and mono changes the stream's format, so receivers have to be set up for it again. Do both before commissioning the receivers, not after.

**Grandmaster changed.** The stream's description names the grandmaster. If a different one takes over, the panel shows an alert until dismissed: receivers set up by hand from the SDP must import it again. Q-SYS in Auto mode picks it up from the announcement.

## 13.5 Lip-sync

Audio sent over the network arrives later than it would through HDMI: the bridge holds about 60 ms between the two clocks, and the sound card another 32 ms. The player allows for this itself, by holding the picture back by the same amount.

What it cannot know is the rest of the chain — the receiver's own buffer, any processing in the Core, and the display's picture delay. Set **Lip-sync offset** in the AES67 panel for those, by eye and ear with a sync test clip (a flash and a beep). Positive values play the audio later; negative, earlier. It applies at once.

## 13.6 Updating

The web interface's update (section 7.1) replaces only the player program. If a release changes the AES67 bridge, run the installer again afterwards: run the one-line installer (which brings the AES67 installer up to date), then press **Update AES67 software** in the AES67 panel and **Run the AES67 installer** — or over SSH:

```
sudo bash /home/pi/pi-video-player/install-aes67.sh
```

On a healthy unit it rebuilds only what changed, in seconds, and never restarts a healthy daemon. After a kernel update it rebuilds everything, and the warnings in 13.3 about the picture apply.

## 13.7 Files, ports and services

| Item | Location or port |
|---|---|
| Services | `aes67-daemon`, `saw-aes67-bridge`, plus `saw-aes67-identity`, `saw-aes67-governor`, `saw-aes67-firewall` (run once at boot) |
| Bridge status, rewritten each second | `/run/saw-aes67/bridge.json` (in memory, not on the SD card) |
| Stereo or mono setting | `/var/lib/saw-aes67/channels` |
| Offset and channels | `/home/pi/config.json` |
| Daemon configuration | `/etc/daemon.conf`; streams kept in `/etc/status.json` |
| Installer's record | `/etc/saw-aes67/` |
| Install from the web interface: log, and outcome | `/var/log/saw-aes67-install.log`, `/var/log/saw-aes67-install.result` |
| The installer copy the web interface runs, and its starter | `/usr/local/lib/saw-aes67/`, `/usr/local/sbin/saw-aes67-webinstall` (both owned by root, put in place by the one-line installer) |
| Audio stream (RTP) | UDP 5004, multicast to the unit's 239.1.x.y group |

| Item | Location or port |
|---|---|
| PTP | UDP 319 and 320 |
| Stream announcements (SAP) | UDP 9875 |
| Stream description for RAVENNA receivers (RTSP) | TCP 8854 |
| Daemon's own web interface and API | TCP 8080 (8081 if something else held 8080 at install — see `/etc/saw-aes67/installed`) — **this Pi only**; blocked from the network, because it has no password |

## 13.8 Fault-finding

```
sudo bash /home/pi/pi-video-player/install-aes67.sh --check   # overall state
cat /run/saw-aes67/bridge.json                                # the bridge's view
curl -s http://127.0.0.1:8080/api/ptp/status                  # PTP lock and grandmaster
curl -s http://127.0.0.1:8080/api/sources                     # the stream as configured
journalctl -u saw-aes67-bridge -u aes67-daemon -b --no-pager | tail -100
cat /proc/asound/RAVENNA/pcm0p/sub0/status                    # RUNNING, with hw_ptr moving
```

| Symptom | Check in this order |
|---|---|
| **"On air" but the receiver hears nothing** | The receiver's Details say `connected=0` → rename the stream (13.4, "A receiver that shows the stream but plays nothing") → IGMP querier on the switches → receiver set to the right group and port → PTP domain → try **Manual** mode with the details from 13.4 |
| **The panel says the daemon restarted** | A receiver may need the stream renamed (13.4). Then dismiss the alert. `journalctl -u aes67-daemon -b` says why it restarted. |
| **Stream not listed in Q-SYS Auto mode** | Announcements are multicast: same VLAN, and the switch passes 239.255.255.255 |
| **Receiver reports a grandmaster mismatch** | The receiver's SDP is older than a grandmaster change — import it again |
| **Robotic or distorted sound** | Kernel 6.15 or later (`uname -r`) → CPU governor not *performance* → CPU temperature |
| **Short gaps every few minutes** | The panel's clock alert. Check the grandmaster is stable, and NTP on the Pi. |
| **Audio leads or lags the picture** | Lip-sync offset (13.5) |
| **AES67 not offered in Audio Output** | The installer has not finished on this unit — run it with `--check` |
| **"Installing from here is not set up on this unit yet"** | Run the one-line installer (section 6.1) again, then reload the page |
| **Install from the web interface "cut off before it finished"** | A power cut or reboot mid-build. Run it again; it carries on from what is already done. |

## 13.9 HTTP API

| Method | Endpoint | Purpose |
|---|---|---|
| `GET` | `/aes67/status` | State, details and alerts as the panel shows them. `{"installed": false}` on a unit without AES67. |

| Method | Endpoint | Purpose |
|---|---|---|
| `POST` | `/aes67/settings` | Body `{"channels": 2, "offset_ms": 0}` — channels 1 or 2, offset −1000 to 1000 |
| `GET` | `/aes67/sdp` | The stream's SDP |
| `POST` | `/aes67/dismiss` | Clear the grandmaster-changed and daemon-restarted alerts |
| `GET` | `/aes67/install` | Whether installing from here is set up, the mode it would use (`keep` or `dark`), and the last install's state and log. Add `?check=1` for the checks — this contacts GitHub. |
| `POST` | `/aes67/install` | Body `{"confirm": true}`. Runs the checks again, then starts the installer. |

Choose AES67 as the output through `/audio_device` with the value `aes67`.

### 13.10 Limitations

- **One stream per unit**, stereo or mono. There is no receiving.
- **The kernel is held.** Operating-system updates leave the kernel alone; after deliberately upgrading it, run the installer again.
- **No authentication** — as for the rest of the web interface (section 12), anyone who can reach port 5000 can change the stream's settings, and start the AES67 install on a unit that does not have it.

## 14. Bluetooth audio

> **Applies to v12 and later.** Proven on the bench (October 2026) with a real speaker: scan, pair, use, the speaker switched off and on, a reboot, the once-a-minute retries, turning Bluetooth off, and forgetting the speaker.
>
> **Check the power supply first.** On the bench, a Pi 5 on a weak supply switched itself off the moment a scan started — the radio's load was the last straw. Use the official 5 V / 5 A supply; from v13 the System panel warns of under-voltage.

Sends the player's audio to a Bluetooth speaker or headphones instead of HDMI. The video still plays on HDMI.

### 14.1 Before you start

- The Bluetooth software is installed by the one-line installer on every unit, but **Bluetooth stays off until somebody turns it on** in the web interface.
- Use **Raspberry Pi OS Lite**. The desktop version runs PipeWire, which takes over Bluetooth audio itself; the panel warns if it is running.
- Bluetooth adds a delay to the sound — typically a tenth to a quarter of a second. The player allows for what the speaker reports, but check lip-sync on site.

### 14.2 Setting it up

In the **Bluetooth Audio** panel:

1. **Turn Bluetooth on.** The Pi stays invisible to phones and other devices: it finds speakers, they do not find it.
2. Put the speaker or headphones in **pairing mode**, then **Scan for devices**. The scan takes 15 seconds. Speakers and headphones are listed first; devices with no name that do not look like audio are left out. What the scan found stays listed for **5 minutes**, then clears — scan again if you need it back.

3. **Pair** the one you want. The Pi pairs with it, trusts it — which is what lets it reconnect after a reboot — and connects. Devices that ask to confirm a code are confirmed automatically; a device that wants a PIN typed in cannot be paired from here.
4. **Use for audio.** The player switches its sound to the speaker. The picture blinks once while it does, as with any change of audio output. The speaker also appears under **Audio Output** (section 4.8), and choosing it there does the same.

## 14.3 What happens on its own

- **After a reboot** the speaker reconnects and plays without anyone touching the web interface, as long as it is switched on and in range.
- Only the speaker chosen **for audio** is reconnected. Turning Bluetooth on does not by itself connect a paired speaker that is not the audio output.
- **If the speaker drops out** — switched off, out of range, flat battery, taken by a phone — the picture keeps playing in silence and the player keeps trying to reconnect: **every few seconds for the first 20 attempts, then once a minute**, for as long as it takes. When it comes back, so does the sound, normally without the picture stopping.
- The panel always says which of these it is doing. **Reconnect now** skips the wait — useful once you have brought the speaker back.
- While the speaker is away the exhibit is silent. It does not switch to HDMI by itself, as that may be a screen with speakers nobody meant to use.

## 14.4 Turning it off, forgetting a device

- **Turn Bluetooth off** switches the radio off. If a speaker was the audio output, the sound goes back to whatever output was used before it — normally HDMI. It stays off across reboots until turned on again.
- **Forget** unpairs a device. If it was the audio output, the sound goes back to the previous output first. To use it again, pair it again.

## 14.5 What the panel says

| The panel says | Meaning and what to do |
|---|---|
| **Playing to …** | All well. |
| Starting Bluetooth… | For the first seconds after the player starts, before it has looked. |
| … dropped out — reconnecting | It has just gone; the first attempt is under way. |
| … is not connected — reconnecting (attempt n of 20) | It dropped out; the player is retrying every few seconds. |
| … has not come back after n attempts — trying once a minute | Check it is switched on, in range, and not connected to a phone. **Reconnect now** to try immediately. |
| No Bluetooth device is the audio output | Bluetooth is on but nothing is using it: pair a speaker and choose **Use for audio**. |
| Bluetooth is blocked on this Pi (rfkill) | `sudo rfkill unblock bluetooth` |
| The Bluetooth audio service is not running | `sudo systemctl status bluealsa` |
| No Bluetooth adapter found | The Pi's Bluetooth is disabled in its configuration, or there is no adapter. |
| The Bluetooth audio software is not installed | Run the one-line installer (section 6.1). |

## 14.6 Fault-finding

```
bluetoothctl show                              # adapter: Powered, PowerState
bluetoothctl devices Paired
bluetoothctl info <MAC>                        # Paired, Trusted, Connected
systemctl status bluetooth bluealsa --no-pager
aplay -L | grep -i -A1 bluealsa                # the speaker as an ALSA device, when connected
journalctl -u saw-player -b | grep -i bluetooth
```

The player's log records changes only — connected, lost, slowing to once a minute, an hourly "still not reachable", back — not every attempt, so a speaker left switched off for a week does not fill the SD card.

## 14.7 Files and API

| Item | Location |
|---|---|
| Bluetooth on or off, and the output to go back to | `/home/pi/config.json` (`"bluetooth"`) |
| Which speaker is the output | `/home/pi/.mpv_audio_device`, like any output |
| Pairing keys | `/var/lib/bluetooth/` (BlueZ's own) |
| Sudo rule for unblocking the radio | `/etc/sudoers.d/saw-bluetooth` |

| Method | Endpoint | Purpose |
|---|---|---|
| `GET` | `/bluetooth/status` | Adapter, paired devices, last scan, link state, what the panel says |
| `POST` | `/bluetooth/enable` | Body `{"enabled": true}` or `{"enabled": false}` |
| `POST` | `/bluetooth/scan` | Look for devices for 15 seconds |
| `POST` | `/bluetooth/pair` | Body `{"mac": …}` — the device's address, as the scan lists it: pair, trust, connect |
| `POST` | `/bluetooth/use` | Body `{"mac": …}`: make it the audio output |
| `POST` | `/bluetooth/forget` | Body `{"mac": …}`: unpair |
| `POST` | `/bluetooth/reconnect` | Try now, and go back to the quick retries |

Actions are queued and answered at once; `/bluetooth/status` reports how they went. One runs at a time.

*Stage Audio Works — Raspberry Pi Video Player, v16. This software is provided with no warranty and no support. See* `README.md`.